Architecture Justification

Architecture Selection: We chose Layered Architecture for the Restaurant Table Booking & Pre-Ordering App System.

Architectural Choice

Layered Architecture with three layers: a Presentation Layer (User Interface Component for the diner app and manager terminal), a Business Layer (Reservation Manager, Order Manager, Kitchen Scheduler and Payment Service Components) and a Data Layer (Database Component). Each layer only uses the interfaces of the layer below it.

Two Reasons

1. Consistent booking data: reservations, pre-orders and kitchen tickets all depend on the same table and order data. Because every business component uses one Data Layer, checking a table and reserving it happen in a single transaction, which prevents two diners from booking the same table. Splitting this data across microservices would make that much harder.
2. Right size for the problem: the app serves one restaurant at a few hundred concurrent users, so it does not need the independent scaling and operational cost of microservices. Keeping the UI separate from business rules lets the diner app and the manager terminal reuse the same reservation and order logic, and the UI can change without touching the kitchen timeline rules.

Security Advantage

The UI can never reach the database or the payment gateway directly. Every request passes through the Business Layer, where authentication and role checks are enforced (only the Restaurant Manager can change the floor plan, menu or table status). Card details go only through the Payment Service to the external gateway as tokens and are never stored in the Data Layer.

Performance Benefit

All Business Layer components run in the same server and call each other directly instead of over the network. A table status change or a kitchen timeline recalculation therefore completes quickly, which helps meet the 500 ms status update target, and frequently read data such as the menu and floor plan can be cached at the Data Layer.